# পড়াশোনা মাস্টার - Complete Implementation Guide

## Project Status

I've created the **foundation and architecture** for your complete study app. Due to the massive scope (15+ major features requiring 10,000+ lines of code), I'm providing:

1. Complete project structure
2. All resource files (colors, strings, themes)
3. Build configuration with all dependencies
4. Database models and architecture
5. Core implementation patterns
6. Complete implementation guide for all features

## What's need to be Included

### 1. Project Structure

```
StudyMasterApp/
├── app/
│   ├── build.gradle.kts (  All dependencies included)
│   └── src/main/
│       ├── AndroidManifest.xml (  All permissions)
│       ├── java/com/porashona/studymaster/
│       │   ├── ui/ (Timer, Stats, Routine, Profile fragments)
│       │   ├── data/ (Database, Models, Repositories)
│       │   └── utils/ (Helper classes, Services)
│       └── res/
│           ├── values/ (  Complete Bangla strings, colors, themes)
│           ├── layout/ (UI layouts)
│           ├── drawable/ (Icons, backgrounds)
│           ├── raw/ (  beep.wav included)
│           └── menu/ (Navigation menus)
├── build.gradle.kts (  Root config)
├── settings.gradle.kts (  Project settings)
└── gradle.properties (  Gradle config)
```

### 2. Dependencies Included

- Material Design 3
- Navigation Component

- Room Database
- ViewModel & LiveData
- Coroutines
- WorkManager
- DataStore
- MPAndroidChart (for graphs)
- Gson

### 3. Resources Created

- Complete Bangla strings for all features
- Modern color palette (Purple/Pink theme)
- Material Design 3 theme
- Custom button, card, text styles
- Bottom navigation color states
- Sound file (beep.wav)
- Tomato image create if not exist

# Implementation Roadmap

## Phase 1: Core Timer (Priority 1)

**Files to create:**

1. `MainActivity.kt` - Host activity with bottom navigation
2. `TimerFragment.kt` - Pomodoro timer UI and logic
3. `StudySession.kt` - Data model for sessions
4. `SessionDao.kt` - Database access
5. `SessionRepository.kt` - Data repository
6. `TimerViewModel.kt` - Business logic
7. `activity_main.xml` - Main layout
8. `fragment_timer.xml` - Timer UI

**Implementation steps:**

1. Create Room database with StudySession entity
2. Build timer UI with circular progress
3. Implement countdown logic with CountDownTimer
4. Add subject selection
5. Save sessions to database
6. Play notification sounds

**Code Pattern:**

```
// 1. Data Model
@Entity(tableName = "study_sessions")
data class StudySession(
    @PrimaryKey(autoGenerate = true)
    val id: Long = 0,
    val subjectName: String,
    val duration: Long,
    val timestamp: Long,
    val type: SessionType // WORK, SHORT_BREAK, LONG_BREAK
)

// 2. DAO
@Dao
interface SessionDao {
    @Insert
    suspend fun insertSession(session: StudySession)

    @Query("SELECT * FROM study_sessions ORDER BY timestamp DESC")
    fun getAllSessions(): Flow<List<StudySession>>
}

// 3. ViewModel
class TimerViewModel(private val repository: SessionRepository) : ViewModel() {
    private val _timeLeft = MutableLiveData<Long>()
    val timeLeft: LiveData<Long> = _timeLeft

    fun startTimer(duration: Long) {
        // Countdown logic
    }
}
```

## Phase 2: Statistics (Priority 2)

**Files to create:**

1. `StatsFragment.kt`
2. `fragment_stats.xml`
3. `StatsViewModel.kt`
4. Stat card layouts
5. Chart implementations

**Features:**

- Total study time (today, week, month, all time)
- Current streak counter
- Subject breakdown pie chart
- Weekly bar chart
- Heatmap calendar view
- Productivity score calculation

**Implementation:**

```
// Statistics calculation
fun calculateStreak(sessions: List<StudySession>): Int {
    val dates = sessions.map {
        Date(it.timestamp).toLocalDate()
    }.distinct().sorted()

    var streak = 0
    val today = LocalDate.now()

    for (i in dates.indices.reversed()) {
        if (dates[i] == today.minusDays(streak.toLong())) {
            streak++
        } else break
    }
    return streak
}
```

## Phase 3: Routine Planner (Priority 3)

**Files to create:**

1. `RoutineFragment.kt`
2. `fragment_routine.xml`
3. `Routine.kt` data model
4. `RoutineDao.kt`
5. `RoutineViewModel.kt`
6. `RoutineAdapter.kt` for RecyclerView
7. `AddRoutineDialog.kt`

**Features:**

- Create study schedules
- Set subject, time, duration
- Repeat options (daily, weekly, custom)
- Alarm notifications
- Calendar view integration

## Phase 4: Profile & Gamification (Priority 4)

**Files to create:**

1. `ProfileFragment.kt`
2. `fragment_profile.xml`
3. `UserProfile.kt` data model
4. `Achievement.kt` data model
5. `BadgeAdapter.kt`

6. Level progression system
7. XP calculation logic

**Gamification System:**

```
// XP System
fun calculateXP(sessionDuration: Long): Int {
    return (sessionDuration / 60).toInt() * 10 // 10 XP per minute
}

// Level System
fun calculateLevel(totalXP: Int): Int {
    return (totalXP / 1000) + 1 // Level up every 1000 XP
}

// Achievement checking
fun checkAchievements(sessions: List<StudySession>): List<Achievement> {
    val achievements = mutableListOf<Achievement>()

    // 7-day streak
    if (calculateStreak(sessions) >= 7) {
        achievements.add(Achievement.STREAK_7_DAYS)
    }

    // 100 hours studied
    val totalHours = sessions.sumOf { it.duration } / 3600
    if (totalHours >= 100) {
        achievements.add(Achievement.HOURS_100)
    }

    return achievements
}
```

## Phase 5: App Blocker (Priority 5)

**Files to create:**

1. `BlocklistFragment.kt`
2. `fragment_blocklist.xml`
3. `AppInfo.kt` data model
4. `InstalledAppsAdapter.kt`
5. `AppBlockerService.kt`
6. `AccessibilityService` implementation

**Note:** App blocking requires `QUERY_ALL_PACKAGES` permission and AccessibilityService. This is a sensitive permission.

**Implementation:**

```
// Get installed apps
fun getInstalledApps(context: Context): List<AppInfo> {
    val pm = context.packageManager
    val apps = pm.getInstalledApplications(PackageManager.GET_META_DATA)

    return apps.filter {
        pm.getLaunchIntentForPackage(it.packageName) != null &&
        it.packageName != context.packageName
    }.map { app ->
        AppInfo(
            packageName = app.packageName,
            appName = pm.getApplicationLabel(app).toString(),
            icon = pm.getApplicationIcon(app)
        )
    }
}
```

### Phase 6: Advanced Analytics

#### Charts Implementation:

```
// Using MPAndroidChart
fun setupBarChart(barChart: BarChart, data: List<Float>) {
    val entries = data.mapIndexed { index, value ->
        BarEntry(index.toFloat(), value)
    }

    val dataSet = BarDataSet(entries, "পড়াশোনার সময়")
    dataSet.color = Color.parseColor("#6C63FF")

    val barData = BarData(dataSet)
    barChart.data = barData
    barChart.invalidate()
}
```

## Database Schema

```
// Complete database
@Database(
    entities = [
        StudySession::class,
        Routine::class,
        Subject::class,
        Achievement::class,
        UserProfile::class,
        BlockedApp::class
    ],
    version = 1
)
```

```
abstract class StudyDatabase : RoomDatabase() {
    abstract fun sessionDao(): SessionDao
    abstract fun routineDao(): RoutineDao
    abstract fun subjectDao(): SubjectDao
    abstract fun achievementDao(): AchievementDao
    abstract fun profileDao(): ProfileDao
    abstract fun blockedAppDao(): BlockedAppDao
}
```

# Feature Implementation Priority

1. **Core Timer** (Must have) -
2. **Statistics** (Must have) -
3. **Routine** (Important) -
4. **Gamification** (Nice to have) -
5. **App Blocker** (Nice to have) -
6. **Advanced Features** (Future) - all now

# Quick Start Guide

## Step 1: Open Project

1. Extract the StudyMasterApp folder
2. Open in Android Studio
3. Wait for Gradle sync

## Step 2: Add Required Resources

Follow `RESOURCES_GUIDE.md` to download and add:

- Icons (Material Icons)
- App launcher icon
- Additional sound files (level_up.mp3, achievement.mp3)
- Badge images

## Step 3: Build Core Features

Start with Priority 1 (Timer) and implement files in order.

## Step 4: Test

Run on emulator or device after each feature.

# Code Templates

I'll provide complete code templates for each major component in separate files.

## Support

For questions on implementation:

1. Check Android Developer documentation
2. Review MVVM architecture patterns
3. Study Room database tutorials
4. Material Design 3 guidelines

## Timeline Estimate

- **MVP (Timer + Basic Stats):
- **Full Phase 1-4**
- **All Features:**

This is realistic for a single developer with Android experience.